

vector

`vector<double> V` 定义了一个名为 `V` 的 `vector` 对象，其中存储的元素类型为 `double`。换句话说，`V` 是一个可以动态增长的双精度浮点数数组。您可以向 `V` 中添加、删除和访问 `double` 类型的元素，并且 `V` 会自动处理内存分配和释放，以确保数组大小的动态调整。

capacity and size

在 C++ 的 `vector` 类中，`capacity` 和 `size` 是两个重要的成员函数，用于描述 `vector` 的当前状态。

1. **capacity:** `capacity()` 函数返回 `vector` 的当前存储空间的容量，即在重新分配内存之前 `vector` 可以存储的元素个数。注意，`capacity()` 返回的是当前内部数组的大小，而不是 `vector` 中实际存储的元素个数。当 `size()` 超过 `capacity()` 时，`vector` 将重新分配内存，增大存储空间，但不会改变 `capacity()` 返回的值。
2. **size:** `size()` 函数返回 `vector` 中实际存储的元素个数，即当前 `vector` 中元素的数量。当向 `vector` 添加元素时，`size()` 会增加；当从 `vector` 删除元素时，`size()` 会减少。

综合来说，`capacity()` 表示 `vector` 内部当前分配的存储空间大小，而 `size()` 表示 `vector` 实际存储的元素数量。

C++

```
1  #include <iostream>
2  #include <vector>
3  using namespace std;
4  int main() {
5      vector<int> vec;
6      // 向 vector 中添加元素
7      for (int i = 0; i < 10; ++i) {
8          vec.push_back(i);
9          cout << "Size: " << vec.size() << ", Capacity: " << vec.capacity() << endl;}
10     return 0;}
```

`vec.push_back(i)` 是 `vector` 容器的成员函数，用于在容器的尾部添加一个新的元素 `i`。具体来说，它会将元素 `i` 添加到 `vector` 的末尾，并将 `vector` 的大小（即存储的元素数量）增加 1。如果此时 `vector` 的存储空间不足以容纳新元素，它会自动分配更多的内存以适应新的元素。

`#include <iomanip>`

iostream

`std::cout << std::fixed << std::setprecision(2) << num << std::endl;`

`std::cout`: 标准输出流

`std::cin`: 标准输入流

`std::cerr`: 标准错误流

`std::endl`: 插入换行符并刷新输出缓冲区

`std::setw()`: 设置输出的字段宽度

`std::setprecision()`: 设置输出流的精度

vector

`std::vector`: 提供了动态数组的功能

`std::vector::push_back()`: 将元素添加到向量的末尾

`std::vector::pop_back()`: 移除向量末尾的元素

`std::vector::size()`: 返回向量中元素的数量

`std::vector::clear()`: 清空向量中的所有元素

string

std::string: 提供了字符串的功能

std::string::size(): 返回字符串的长度

std::string::append(): 将字符串添加到当前字符串的末尾

std::string::substr(): 返回从指定位置开始的子字符串

std::string::find(): 在字符串中查找指定的子字符串

algorithm

std::sort(): 对容器中的元素进行排序

std::find(): 在容器中查找指定元素

std::max(): 返回两个值中的最大值

std::min(): 返回两个值中的最小值

std::count(): 计算容器中指定值的出现次数

cmath

std::abs(): 计算绝对值

std::sqrt(): 计算平方根

std::pow(): 计算指数值

std::sin(), std::cos(), std::tan(): 计算三角函数值

fstream

std::ifstream: 用于从文件中读取数据的输入流

std::ofstream: 用于向文件中写入数据的输出流

std::fstream: 用于同时读写文件的输入输出流

std::getline(): 从输入流中读取一行数据

`<cstdlib>` 是 C++ 标准库中的一个头文件，提供了一系列常用的函数和类型，用于各种操作，包括内存分配、转换和随机数生成等。常见的用法包括：

1. 内存分配和释放：

- `malloc`、`calloc`、`realloc`、`free`

2. 整数转换函数：

- `atoi`、`atol`、`atoll`、`strtol`、`strtoll`
- `atof`、`strtod`

3. 伪随机数生成：

- `rand`、`srand`

4. 其他实用函数：

- `exit`、`abort`
- `system`

在程序开头包含 `<cstdlib>` 头文件可以让你使用这些功能而无需自己定义它们。

C++

```
1  #include <iostream> // 包含输入输出流库
2  #include <string>   // 包含字符串库
3  using namespace std; // 使用 std 命名空间
4  // 定义一个结构体表示学生
5  struct Student {
6      string name;
7      int age;
8      float gpa;
9  };
10 // 函数重载示例
11 void print(int num) {
```

```

12     cout << "Integer: " << num << endl;
13 }
14
15 void print(float num) {
16     cout << "Float: " << num << endl;
17 }
18
19 // 函数声明，带有默认参数
20 int add(int a, int b = 0);
21
22 // 函数定义，实现加法
23 int add(int a, int b) {
24     return a + b;
25 }
26
27 int main() {
28     // 变量声明和赋值
29     int x = 10;
30     float y = 3.14;
31
32     // 运算符示例
33     cout << "Sum: " << add(x, 5) << endl; // 调用 add 函数
34     print(x); // 调用函数重载的 print 函数
35     print(y);
36
37     // 字符串示例
38     string message = "Hello, World!";
39     cout << message << endl;
40
41     // 数组和指针示例
42     int arr[3] = {1, 2, 3};
43     int* ptr = arr; // 数组名是一个指向数组第一个元素的指针
44     cout << "Array elements: ";
45     for (int i = 0; i < 3; ++i) {
46         cout << *ptr << " "; // 使用指针访问数组元素
47         ptr++;
48     }
49     cout << endl;
50
51     // 结构体示例
52     Student s;
53     s.name = "Alice";
54     s.age = 20;
55     s.gpa = 3.5;
56     cout << "Student: " << s.name << ", Age: " << s.age << ", GPA: " << s.gpa << endl;
57
58     // 控制语句示例
59     if (x > 5) {
60         cout << "x is greater than 5" << endl;
61     } else {
62         cout << "x is not greater than 5" << endl;
63     }
64
65     // 异常处理示例
66     try {
67         throw runtime_error("An error occurred!"); // 抛出异常
68     } catch (const exception& e) {
69         cout << "Exception caught: " << e.what() << endl; // 捕获异常并打印错误消息
70     }
71
72     return 0;
73 }

```

常函数，常变量